

PyNotebook

SYSTEM DESIGN SPECIFICATION DOCUMENT

Chris O'Brien, Andrew Presta, Vincent Tran
DESALES UNIVERSITY | 2755 STATION AVENUE, CENTER VALLEY, PA 18034

Software Design Document Specification Template

The Software Design Specification (SDS) sections provide you with guidelines related to the structure and the contents of SDS document. The Software Design Specification document includes at least these sections.

For the project, your team may have good reasons for wanting to deviate from this proposed outline. If a section is not applicable in your case, do not delete it; instead, give the topic heading and write "Not applicable".

You will note that there is some overlap in the content between different documents (i.e. the User Requirements Specification Document and the Software Design Specification Document.) This redundancy allows each document to stand on its own.

**ONLY THE SECTION TITLES COLORED IN ORANGE
ARE REQUIRED TO BE COMPLETED.**

**DO NOT DELETE THE SECTIONS YOU ARE NOT
COMPLETING AS THEY ARE A PART OF THE
DOCUMENT**

Contents

1	Introduction	3
1.1	Purpose of this document.....	3
1.2	Scope of the development project	3
1.3	Definitions, acronyms, and abbreviations	3
1.4	References	3
1.5	Overview of document	3
2	System architecture description.....	3
2.1	Overview of modules / components	3
2.2	Structure and relationships	3
2.3	User interface	3
2.4	User interface issues	3
3	Detailed description of components (ONLY 2 ARE REQUIRED)	3
3.1	X Component (or Class or Function ...)	3
3.2	Y Component (or Class or Function ...)	4
4	Reuse and relationships to other products	4
5	Design decisions and tradeoffs	4
6	Pseudocode for components	4
7	Appendices (if any)	4
	Software component template for section 3	4

1 INTRODUCTION

1.1 PURPOSE OF THIS DOCUMENT

Full description of the main objectives of the SDS document.

1.2 SCOPE OF THE DEVELOPMENT PROJECT

This will be similar to what was written in the SRS.

1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS

Be sure to alphabetize!

1.4 REFERENCES

This section will include technical books and documents related to design issues. Be certain that the references you give are complete and in the appropriate format.

1.5 OVERVIEW OF DOCUMENT

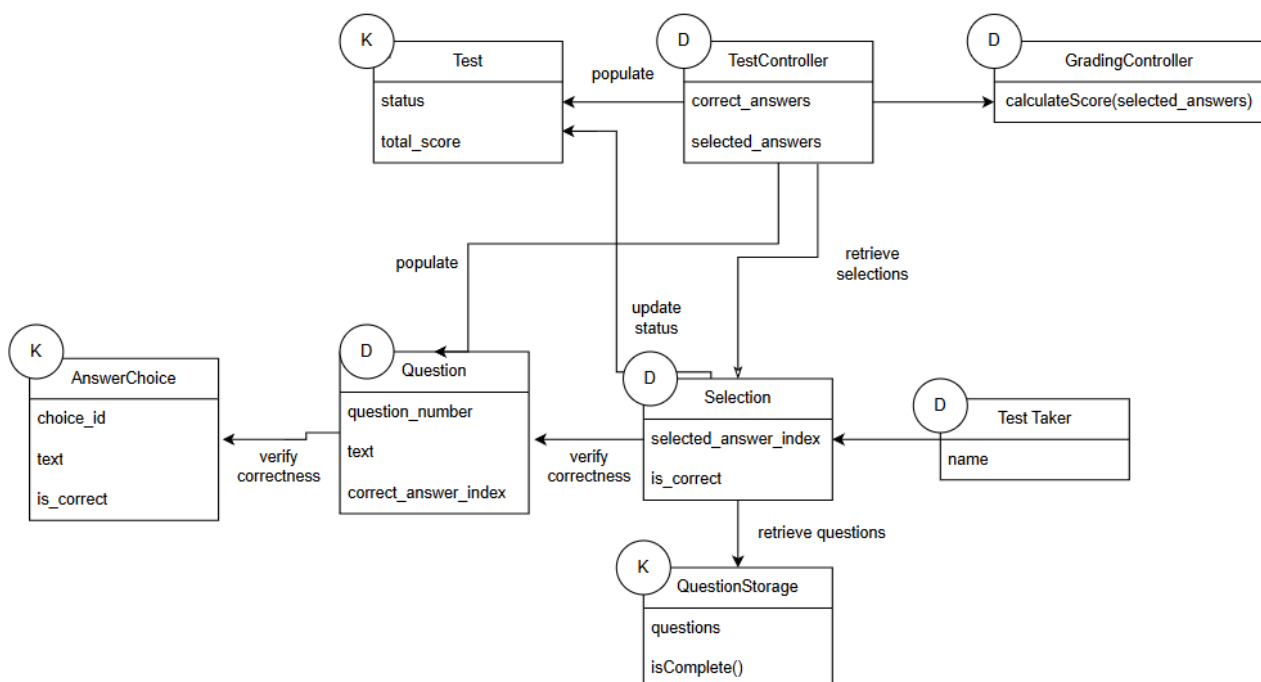
A short description of how the rest of the SDS is organized and what can be found in the rest of the document. This is not simply a table of contents. Motivate and briefly describe the various parts!

2 SYSTEM ARCHITECTURE DESCRIPTION

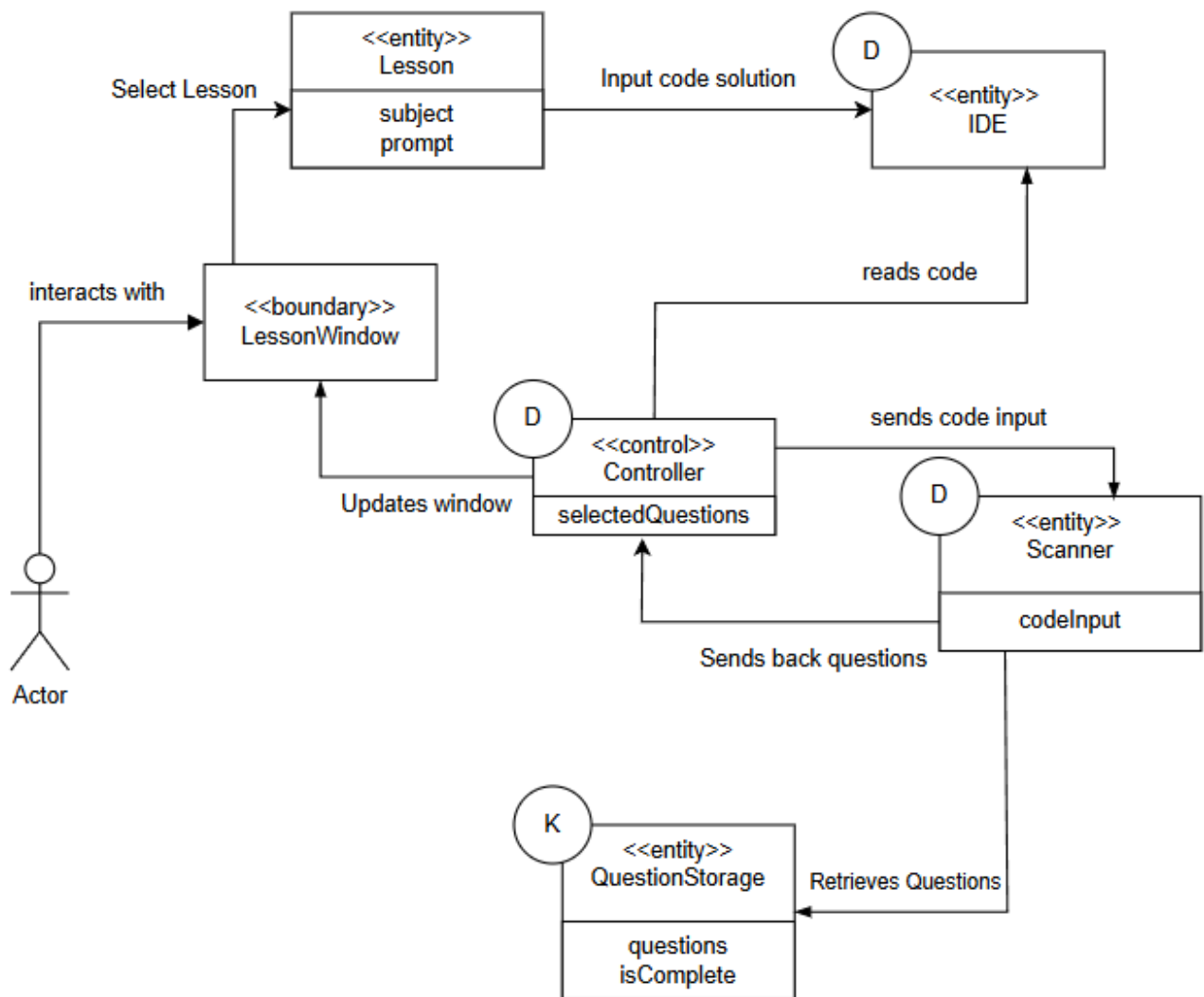
2.1 OVERVIEW OF MODULES / COMPONENTS

This subsection will introduce the various components and subsystems.

Use case 1



Use case 4



2.2 STRUCTURE AND RELATIONSHIPS

2.2.1 Responsibilities

Responsibility	Type	Concept Name
Rs1. Controlling the inputs from the user base to define/find questions for the quiz	D	Controller
Rs2. Scanning in lines of code from the IDE to pull questions from storage to be sent to the controller	D	Scanner
Rs3. The storage of all available questions to be pulled based on information found by the scanner	K	Question Storage
Rs4. Window of code input to be sent to the controller for use by the scanner	D	IDE
Rs5. Selection of answers from the user to complete the quiz functionality	D	Selection

2.2.2 Concept Pairs and Associations

Concept Pair	Association Description	Association Name
Controller <-> IDE	The controller reads the code input given to the IDE by the user	Reads code
Lesson <-> IDE	The lesson entity is given the correct coding solution by the user	Inputs code solution
Controller <-> Scanner	The Controller will send necessary code input to the scanner to be scanned for questions	Sends code input
Selection <-> QuestionStorage	Selection will retrieve adequate questions based on the coding categories found within the code	Retrieve questions
TestController <-> Test	The Test controller will populate the test with the correct questions pulled from storage	populate

2.2.3 Concept Attributes

Concept	Attributes	Attribute Description
Question	question_number	The question number (ex. 1, 2, 3, etc)
Question	text	The text of the question being asked
Question	correct_answer_index	Position of the correct answer (0-3)
AnswerChoice	choice_id	The unique identifier indicating answer position (ex. 0 (option A), 1 (option B), etc)
AnswerChoice	text	The text of the answer choice
AnswerChoice	is_correct	Indicates whether this choice is correct
TestController	correct_answers	Contains all of the correct choice_ids
TestController	selected_answers	Contains all of the selected choice_ids
Lesson	subject	The lesson topic (ex. Loops)
Lesson	prompt	The information guiding the user for the lesson
QuestionStorage	questions	Stores all the questions for a given test

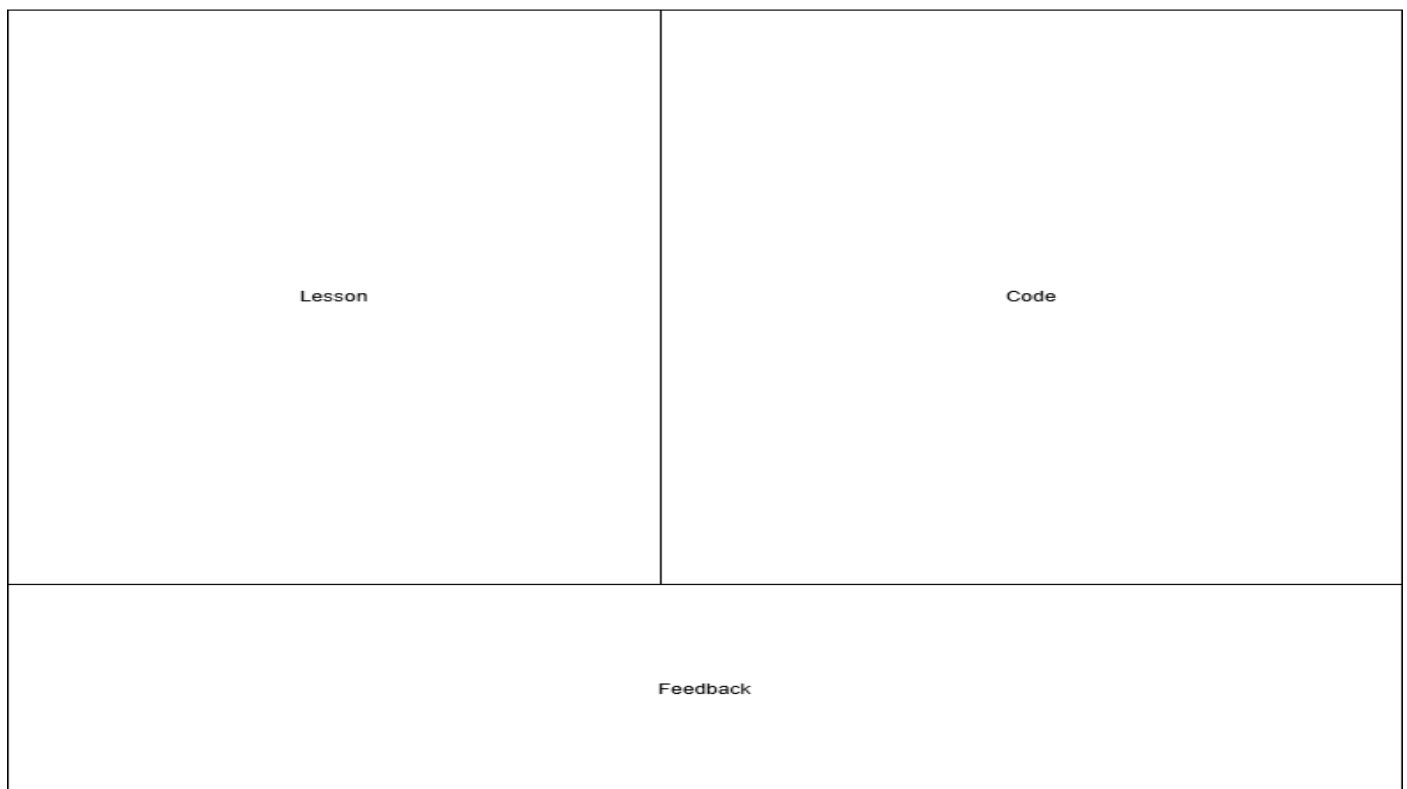
QuestionStorage	isComplete	Stores whether a test has been complete
-----------------	------------	---

2.3 USER INTERFACE

PyNotebook

Lessons
Tests

File	Edit	View	Run	Kernel	Settings	Help
------	------	------	-----	--------	----------	------



2.4 USER INTERFACE ISSUES

This section will present the main principles of the product's user interface. Use the personas/use cases defined in the SRS to make specific examples. This section should not touch on technical details. You may want to include sketches and specific text messages.

- Overarching Design Principles
 - o simplicity, visible progression, immediate feedback
- Main Lesson Window Layout
 - o wireframe showing completion states, locked lessons, and the repeat-lesson flow
- Paste Detection and Manual Input
 - o Recommended dialog wording, and a note on how self-taught users may experience this restriction differently
- Quiz Interface
 - o Wireframe of the questions/answer screen, with notes on the Submit button state
- Results and Feedback Screen
 - o per-question results wireframe tied to use cases

- Error and Edge Case Messaging
 - o A section of all required user-facing messages with plain-language copy suggestions

3 DETAILED DESCRIPTION OF COMPONENTS

3.1 TEST COMPONENT

<u>Identification</u>	Test, located in the core application layer of PyNotebook
<u>Type</u>	Class
<u>Purpose</u>	Implements PyNotebook's assessment functionality. Satisfies multiple-choice test, submission, and feedback, single-page question display, four answer choices, category-based generation, and repeat lesson. Derived requirements include full-answer validation before submission and score persistence to support lesson replay.
<u>Function</u>	Accepts parsed concept categories from the user's submitted code, maps them to relevant multiple-choice questions (four options each), renders the full test on a single page, evaluates submitted answers, displays a score, and writes results to the database for progress tracking.
Subordinates	Question maps parsed categories to questions and answer options. The display renders all questions on one page. Answer captures and records user selections. Selection validates completion, scores the test, and displays feedback. The result persists the score and completion status.
<u>Dependencies</u>	Depends upstream on the In-App IDE (valid code submission required before activation) and the Code Scanner/Category Parser (parsed categories are the primary input). Used downstream by the Lesson Interfacing Component (triggers display transition) and the Progress/Database Layer (receives stored results). The quiz only instantiates after successful parsing, and results must be saved before the user navigates away.
<u>Interfaces</u>	Receives a structured category list from the Code Scanner. Renders a single-page quiz UI with radio-button answer options and a Submit button. Displays error messages for unanswered questions ("Please answer all questions before submitting."), empty generation ("No test available."), and general failures. Outputs score and completion status to the database upon submission.
Resources	Requires a Python runtime, Python parsing/analysis libraries, a GUI framework for rendering, and database/file storage for result persistence.
Processing	On activation, generate questions from parsed categories; display error if none exist. Render quiz and record answer selections as the user interacts. On submission, validate all questions are answered or block with an error message. Evaluate answers, calculate and display score, then persist results to the database.

Data	questions – a list of Question object retrieved from a generated test containing a prompt and answer choices test_id – identifies which test to display. Retrieves the proper set of questions question_count – integer representing the number of questions for a test
------	--

3.2 LESSON COMPONENT

<u>Identification</u>	Lesson, located in the data layer of PyNotebook
<u>Type</u>	Class
<u>Purpose</u>	Stores and provides lesson content to the user interface and links each lesson to its associated test. This satisfies lesson guidelines, lesson interfacing, and repeat lesson. The derived requirements include maintaining a unique lesson to associate test results and completion status, and providing enough metadata to render lesson content without external sources.
<u>Function</u>	Holds a lesson's identifying data and maintains a one-to-one association with a quiz object. On selection by the user, it supplies content to the lesson on the quiz component for result tracking. It acts as the authoritative data record for a given lesson.
Subordinates	The lesson class itself is a flat entity with no internal sub-components. Its constituents are its data fields, which are its unique identifier, the lesson title, and the instructional content rendered.
<u>Dependencies</u>	Depends on the database layer for persistence across sessions. Used by the boundary to display lesson content to the user. Used by the test object, which is associated with the lesson to link quiz status and goes back to the correct lesson. The lesson must be loaded before the window renders and before the quiz can be initialized.
<u>Interfaces</u>	Provides lesson attributes to the lesson window for UI rendering. It Passes the id of the lesson to the test object to establish the lesson-to-quiz association. No direct user interaction occurs with the Lesson object itself. All interaction is mediated through the lesson window. If a lesson fails to load, the system should display an error preventing the window from rendering with missing content.
Resources	Requires database or file storage to retrieve and persist lesson records across sessions. Requires sufficient memory to hold lesson content for display. No special hardware or external libraries are needed beyond the standard Python runtime.
Processing	The user selects a lesson. Then, the lesson loads a record matching the selected lesson id from storage. If the load fails, then display an error and abort the lesson launch. Pass the name and info to the lesson window for rendering, and pass the lesson id to the test to initialize it.
Data	lesson_id – identifies the lesson based on a unique integer value title – string for the lesson name content – string containing the instructional material guidelines – string providing hints for the student Problem_description – string that describes the problem

3.3 SCANNER FUNCTION

<u>Identification</u>	Scanner, located within the input chain of the codebase
<u>Type</u>	A subprogram
<u>Purpose</u>	The design of the scanner function is to function properly given any amount of code input. Our general model should have very limited code bases sent to the program, but the efficiency of how the scanner processes code input is of the highest priority
<u>Function</u>	The goal of the scanner function is to pull basic coding categories out of submitted lines of code. Firstly, the scanner will be sent each line of code individually. With each line, the scanner will search each line for keywords that can trigger a category within our questions bank (i.e. a variable declaration, if, then, etc.) Once the categories are found within the code, the scanner retrieves the adequate questions to be sent to the controller. The goal is to find roughly 3 questions that challenge the user to think more deeply about their solution
Subordinates	Lower on the chain, the Question Storage component works off the Scanner, as it pulls questions based on the found categories.
<u>Dependencies</u>	The main controller and general flow of the entire program is dependent on the scanner function to generate questions. Without the ability of generate testing questions, the user can not make progress through a lesson plan.
<u>Interfaces</u>	The scanner function doesn't have any interfacing tools that are external or shown to the user. As explained above, it will send the appropriate data to the controller for interfacing externally. Internally, the main errors we expect to find are in the inability to read the code input. An error message will be sent to the console for testing.
Resources	A complete description of all resources (hardware or software) external to the component but required to carry out its functions. Some examples are CPU execution time, memory (primary, secondary, or archival), buffers, I/O channels, plotters, printers, math libraries, hardware registers, interrupt structures, and system services.
Processing	Main for line in input: For word in line: If word is in categoryList: Question1 = question #1 from QuestionStorage : word Etc... Functionality:
Data	For the Processing section. The categoryList is most likely a copy of the keys within the QuestionStorage

3.4 QUESTION STORAGE COMPONENT

<u>Identification</u>	Question Storage
<u>Type</u>	A data file/dictionary
<u>Purpose</u>	Purpose of Question storage is to be a file containing a plethora of questions pertaining to each category within our lesson base

<u>Function</u>	It will output questions pulled by the scanner function as described in Component 3.3. It has no other functionality.
Subordinates	The Question Storage itself is a subordinate of the Scanner function and is at the lowest end of the program.
<u>Dependencies</u>	The Scanner function is completely dependent on pulling from question storage to provide questions for the quiz. See the Processing section of 3.3 for more information regarding the utilization of the storage system.
<u>Interfaces</u>	More information will be stated in processing, but in general the storage will hold a key value pair for the scanner to pull from. Keys will contain all categories from lessons, and the values will hold the various questions.
Resources	We expect the question storage will have an extremely small impact on the overall performance of the system. Thus, the component really won't have many resources attributed to it, besides memory. As PyNotebook, creates more lessons, the storage will grow to accept more questions and categories.
Processing	Key Value dictionary Keys: Each category or main topic discussed with the lessons of PyNotebook (variables, loops, if then statements, etc.) Values: A tuple of questions pertaining to those key values, the scanner function will grab a random question from the tuple to pull for the quizzes.
Data	Both key and value pairs will be strings that are described as explained above.

3.5 SELECTION COMPONENT

<u>Identification</u>	Selection, located in the test section of the PyNotebook program.
<u>Type</u>	Class
<u>Purpose</u>	The selection component is made to track user interactions during a test. It ensures that every answer is recorded and can be updated before the test taker submits a test.
<u>Function</u>	The selection component captures the index of the answer chosen by the test taker for a question (selected_answer_index). It will take a click event from the UI and store the index of the selection. Additionally, it will store whether or not the answer selected is correct (is_correct) by comparing with the Question object's correct_answer_index.
Subordinates	selected_answer_index: integer representing the user's choice (0-3) Is_correct: boolean determined after a selection has been made
<u>Dependencies</u>	The selection component uses the Question component to verify the correct index and the QuestionStorage for storing data. It is used by the Test Controller for grading. The selection component is also dependent on the test taker making a selection in the UI. This is modified if the test taker changes their answer.
<u>Interfaces</u>	Each question will have 4 answers choices and a selection component will fill a radio button next to the answer choice. This is a circle that is filled or unfilled next to an answer choice, filled with the answer choice is selected.

Resources	The software that will be used is Python standard library specifically data classes. The component will use primary RAM for memory storage to hold the selection state. This will be stored until the application is closed.
Processing	<pre> If user_clicked(index) Selected_answer_index = index If selected_answer_index == Question.correct_answer_index: Is_correct = True Else: Is_correct = False QuestionStorage.save(answeroption) </pre>
Data	selected_answer_index – Integer representing the answer chosen by the test taker. The value ranges from 0–3, corresponding to the four answer choices. The initial value is None until a selection is made. is_correct – Boolean value indicating whether the selected answer matches the correct answer from the Question component. It is True if the answer is correct and False otherwise.

3.6 TEST CONTROLLER COMPONENT

<u>Identification</u>	TestController, the logic coordinator for the PyNotebook program
<u>Type</u>	Controller
<u>Purpose</u>	Function and performance requirements implemented by the design component, including derived requirements. Derived requirements are not explicitly stated in the SRS, but are implied or adjunct to formally stated SDS requirements. The purpose of the test controller component is to manage the lifecycle of a test. It is responsible for populating a test with questions and answer options, checking if all questions have been answered before the test taker submits, and calculating the final score.
<u>Function</u>	What the component does, the transformation process, the specific inputs that are processed, the algorithms that are used, the outputs that are produced, where the data items are stored, and which data items are modified. The TestController component takes in Python code and generates the test based on the code. It keeps a reference of a Test object and all the Question objects. It also provides the logic for checking isComplete() from the QuestionStorage object.
Subordinates	Correct_answers: integer holding the number of correct answers Selected_answers: A list of Selection objects that changes based on test taker selection.
<u>Dependencies</u>	The TestController component uses a Selection to determine all selected answers and uses a Question to determine all correct answers and compares with GradingController to grade. The controller must also indirectly work with QuestionStorage to retrieve and save data. The controller is used by the UI to process button clicks such as submitting a test.
<u>Interfaces</u>	The TestController component will be in charge of displaying all questions and answer choices for each question, as well as creating the feedback that is displayed to the user after submission. It will also handle any error message displays from pressing submit.

Resources	The TestController component will require access to Lesson data to provide context for generating a test. It will utilize memory to hold information about a Test.
Processing	<pre> Def handle_submit(user_answers): If isComplete(user_answers) FALSE UI displays error message Else: Score = calculateScore(user_answers) Test.total_score = score UI displays feedback </pre>
Data	correct_answers – Integer representing the total number of correctly answered questions. The initial value is 0 and it is updated when the test is graded. selected_answers – A list containing Selection objects representing the answers chosen by the test taker. This list is updated whenever the user selects or changes an answer. test – A reference to the Test object that contains all questions and answer options for the current test. This allows the controller to access questions and compute the final score.

4 REUSE AND RELATIONSHIPS TO OTHER PRODUCTS

For teams doing enhancement work, reuse is an important issue. Most enhancement work should focus on extending, rather than replacing, the design and product development from earlier semesters. For teams doing new development, reuse can also be an important strategy. In some cases, there is freeware that could be incorporated. In other cases, there are existing modules or classes that could be adapted. Another possibility is the use of special tools that produce open source results and thus permissible under the terms of this course.

This section should include the following subsections as appropriate:

- How reuse is playing a role in your product design
- How reuse is playing a role in your product implementation (and the motivation for changes)
- If you are not reusing material that is available, then give motivation for why it is being thrown out.

5 DESIGN DECISIONS AND TRADEOFFS

A major tradeoff we made within our program is in our scanner function. We traded off the efficiency of our scanner speed for the reassurance and recuperation of scan line by line of the user's code. This design decision plays with our philosophy of the original PyNotebook look, to ensure the learning of past lessons through the quizzes.

6 PSEUDOCODE FOR COMPONENTS

Utilize the use cases to create pseudocode for components.

7 APPENDICES (IF ANY)

SOFTWARE COMPONENT TEMPLATE FOR SECTION 3

The template given below suggests a reasonable structure for giving a thorough description of each component described in Part 3 of the SDS. The specific information depends in part on the design approach. Your team must adapt this template to your needs and describe it in section 3.1 of your SDS.

<u>Identification</u>	The unique name for the component and the location of the component in the system.
<u>Type</u>	A module, a subprogram, a data file, a control procedure, a class, etc.
<u>Purpose</u>	Function and performance requirements implemented by the design component, including derived requirements. Derived requirements are not explicitly stated in the SRS, but are implied or adjunct to formally stated SDS requirements.
<u>Function</u>	What the component does, the transformation process, the specific inputs that are processed, the algorithms that are used, the outputs that are produced, where the data items are stored, and which data items are modified.
Subordinates	The internal structure of the component, the constituents of the component, and the functional requirements satisfied by each part.
<u>Dependencies</u>	How the component's function and performance relate to other components. How this component is used by other components. The other components that use this component. Interaction details such as timing, interaction conditions (such as order of execution and data sharing), and responsibility for creation, duplication, use, storage, and elimination of components.
<u>Interfaces</u>	Detailed descriptions of all external and internal interfaces as well as of any mechanisms for communicating through messages, parameters, or common data areas. All error messages and error codes should be identified. All screen formats, interactive messages, and other user interface components (originally defined in the SRS) should be given here.
Resources	A complete description of all resources (hardware or software) external to the component but required to carry out its functions. Some examples are CPU execution time, memory (primary, secondary, or archival), buffers, I/O channels, plotters, printers, math libraries, hardware registers, interrupt structures, and system services.
Processing	The full description of the functions presented in the Function subsection. Pseudocode can be used to document algorithms, equations, and logic.
Data	For the data internal to the component, describes the representation method, initial values, use, semantics, and format. This information will probably be recorded in the data dictionary.